Architecture proposée

Principes directeurs

Conserver Laravel et Docker Compose afin de limiter l'impact sur l'application.

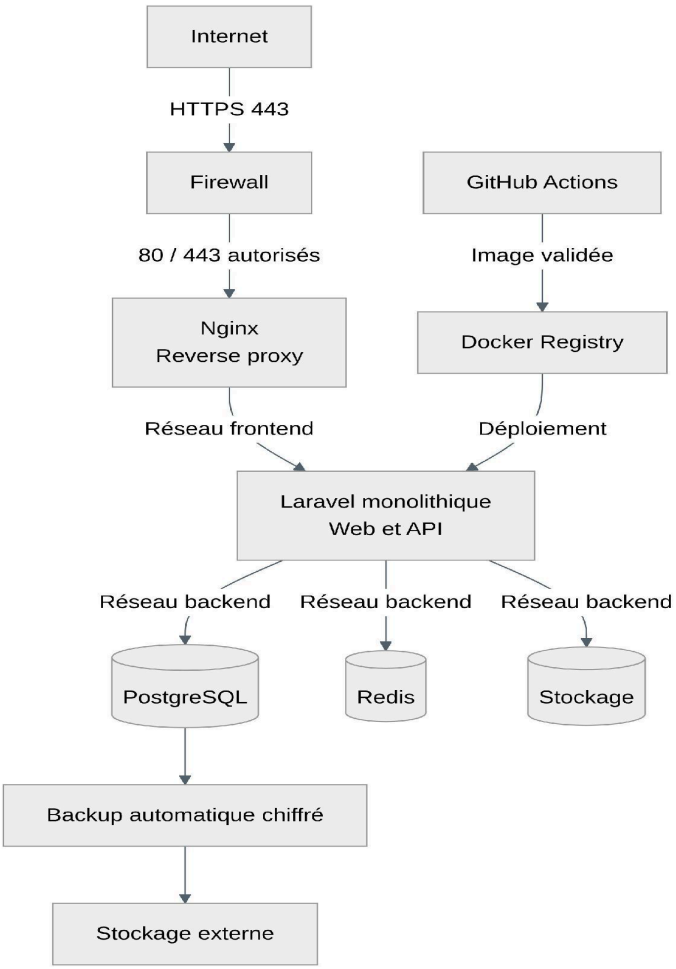
Faire de Nginx l'unique point d'entrée applicatif public.

Ne publier aucun port Laravel, PostgreSQL ou Redis sur Internet.

Cumuler firewall hôte et isolation par réseaux Docker.

Séparer strictement staging et la production.

Automatiser les validations, la construction et le déploiement des images.



Le firewall limite l’exposition du serveur. Nginx termine TLS et transmet les requêtes à Laravel sur le réseau frontend. Laravel accède aux services de données par le réseau backend. Les sauvegardes PostgreSQL sont externalisées et les images proviennent d’une chaîne CI/CD contrôlée.

6. Flux de communication

6.1 Flux utilisateur

Le client atteint Nginx en HTTPS sur le port 443. Nginx applique TLS et les contrôles de première ligne, puis transmet la requête au conteneur Laravel sur le réseau frontend. Laravel traite la requête et accède aux services internes via le réseau backend.

6.2 Flux API externe

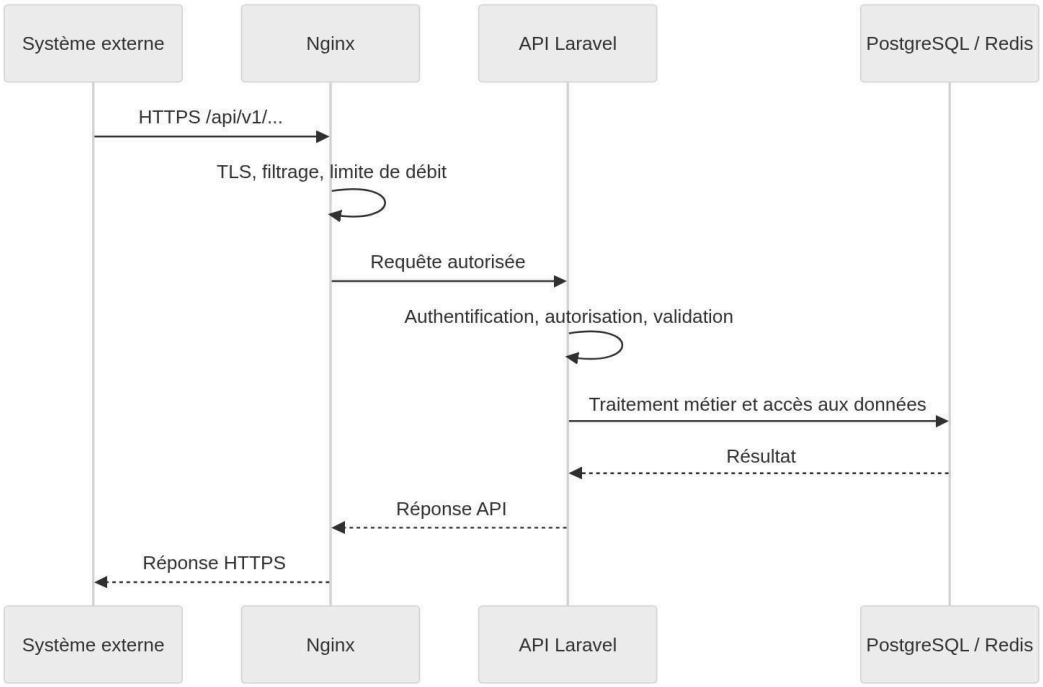


Figure 6 — Traitement d'une requête API externe

Les endpoints externes sont regroupés sous /api/v1/.... Nginx applique les contrôles de transport et de trafic; Laravel réalise les contrôles d'identité, de droits, de validité et le traitement métier avant tout accès aux données.

Sécurité

4.1 Exposition réseau et isolation

La règle fondamentale est qu'aucun service applicatif ou de données ne soit directement accessible depuis Internet. Nginx constitue le seul point d'entrée public de l'application; le firewall apporte une barrière indépendante au niveau de l'hôte.

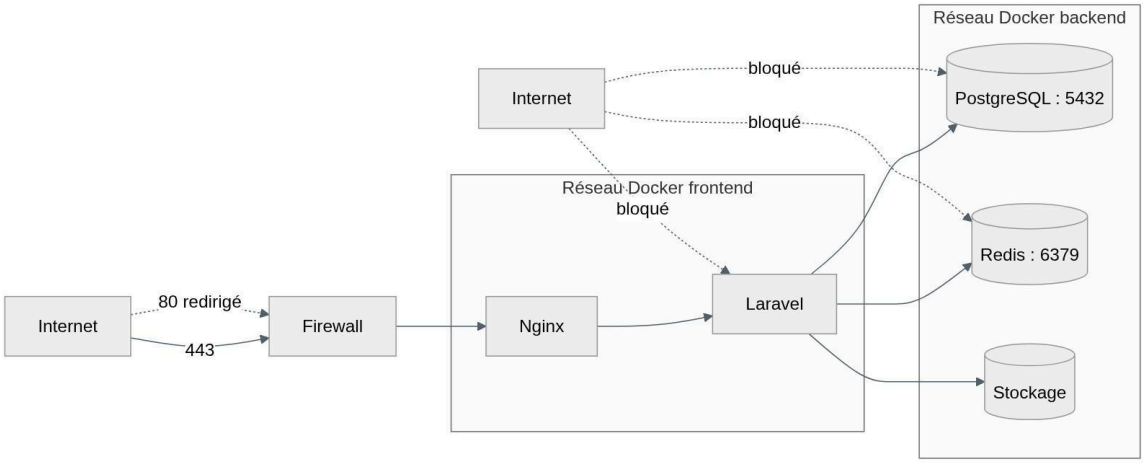


Figure 3 — Segmentation réseau et politique de ports

Nginx et Laravel partagent le réseau frontend. Laravel seul rejoint également le réseau backend qui contient PostgreSQL, Redis et le stockage. Les flèches en pointillé illustrent les accès publics bloqués.

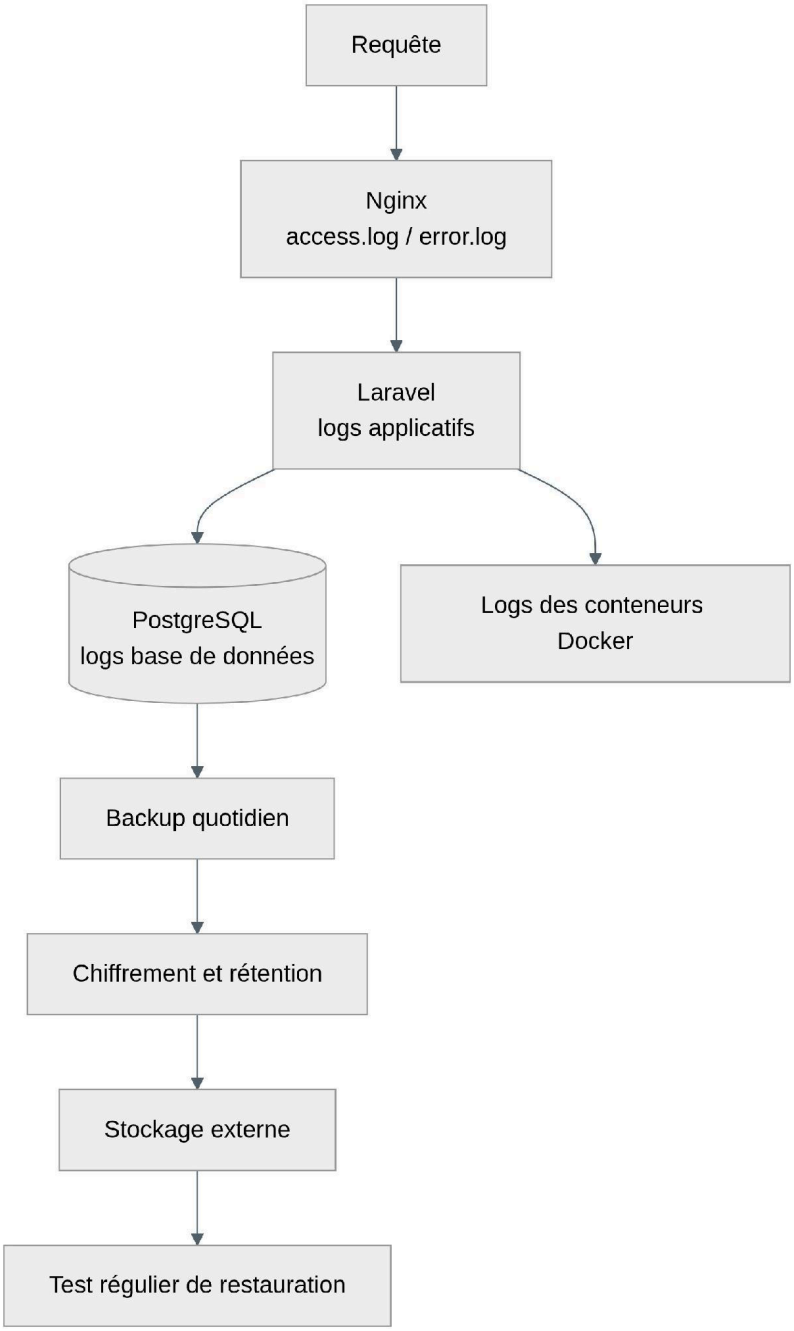
4.2 HTTPS, filtrage et défense en profondeur

- Présenter l'application sur un domaine HTTPS, par exemple distributeur.example.com.
- Rediriger systématiquement HTTP vers HTTPS.
- Configurer dans Nginx les limites de débit, tailles maximales de requête, en-têtes de sécurité, règles de filtrage et logs d'accès.
- Conserver dans Laravel l'authentification, l'autorisation, la validation et les contrôles métier. Nginx complète ces mécanismes mais ne les remplace pas.

4.3 Secrets et séparation des environnements

Chaque environnement doit disposer de ses propres variables, secrets, certificats, stockage, journaux et base de données. Le staging ne doit jamais se connecter à la base de production. Les secrets ne doivent pas être intégrés aux images Docker ni au dépôt source.

4.4 Sauvegardes et journalisation



Les journaux sont produits à chaque couche afin de reconstituer le parcours d’une requête. Les sauvegardes PostgreSQL sont quotidiennes, chiffrées, soumises à une politique de rétention et copiées hors du serveur de production; leur restauration est testée régulièrement.

- Conserver au minimum les logs access/error de Nginx, les logs Laravel, les logs conteneurs et les logs PostgreSQL.
- Prévoir à terme une centralisation des logs pour le monitoring et les investigations.
- Éviter que le serveur de production soit l’unique emplacement des sauvegardes.

Déploiement

5.1 Environnements

Staging

Validation avant production; domaine possible : staging-distributeur.example.com. Base, variables, secrets, stockage, logs, domaine et certificat dédiés; aucun accès à la base de production.

Production

Service réel accessible aux utilisateurs et systèmes externes.
Infrastructure, données, secrets et sauvegardes propres à la production.

5.2 Docker Compose

Docker Compose orchestre Nginx, Laravel, PostgreSQL, Redis et les services associés. Seuls les ports 80 et 443 de Nginx sont publiés. Les services internes utilisent les réseaux frontend et backend selon leur besoin de communication.

5.3 Chaîne CI/CD

